

杭州电子科技大学

实验报告

课程名称：密码学基础与算法课程设计

姓名：xx 学号：xx

实验地点：4 教 441-443-445-447

实验时间：周五 6-8 节

一、实验名称：DES 密码实验

二、实验要求：

- 1、了解分组密码的起源与涵义。
- 2、掌握 DES 密码的加解密原理。
- 3、用 Visual C++实现 DES 密码程序并输出结果。

三、实验内容：

1、1949 年，Shannon 发表了《保密系统的通信理论》，奠定了现代密码学的基础。他还指出混淆和扩散是设计密码体制的两种基本方法。扩散指的是让明文中的每一位影响密文中的许多位，混淆指的是将密文与密钥之间的统计关系变得尽可能复杂。而分组密码的设计基础正是扩散和混淆。在分组密码中，明文序列被分成长度为 n 的元组，每组分别在密钥的控制下经过一系列复杂的变换，生成长度也是 n 的密文元组，再通过一定的方式连接成密文序列。

2、DES 是美国联邦信息处理标准(FIPS)于 1977 年公开的分组密码算法，它的设计基于 Feistel 对称网络以及精心设计的 S 盒，在提出前已经进行了大量的密码分析，足以保证在当时计算条件下的安全性。不过，随着计算能力的飞速发展，现如今 DES 已经能用密钥穷举方式破解。虽然现在主流的分组密码是 AES，但 DES 的设计原理仍有重要参考价值。在本实验中，为简便起见，就限定 DES 密码的明文、密文、密钥均为 64bit，具体描述如下：

明文 m 是 64bit 序列。

初始密钥 K 是 64 bit 序列(含 8 个奇偶校验 bit)。

子密钥 $K_1, K_2 \dots K_{16}$ 均是 48 bit 序列。

轮变换函数 $f(A, J)$ ：输入 A (32 bit 序列), J (48 bit 序列)，输出 32 bit 序列。

密文 c 是 64 bit 序列。

1) 子密钥生成:

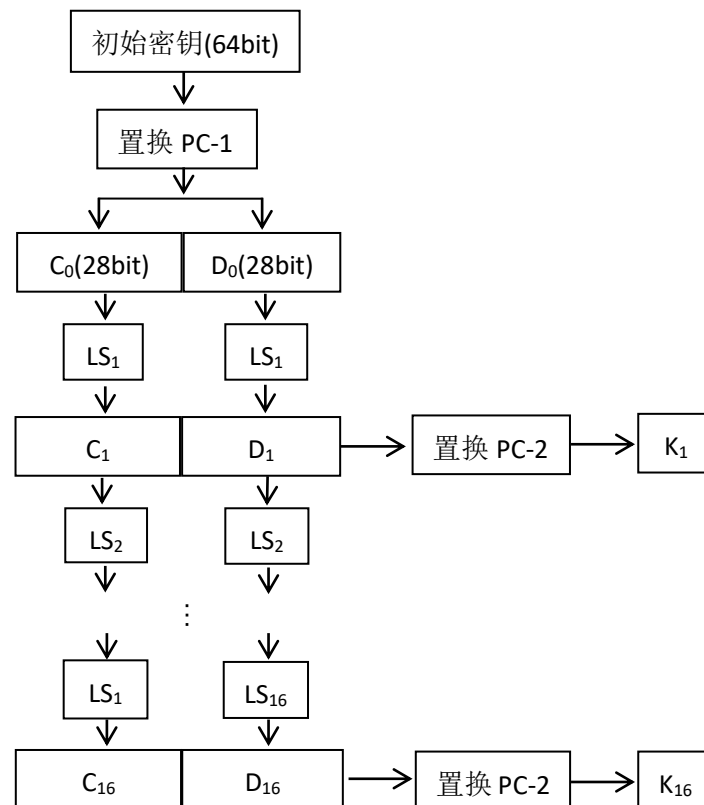
输入初始密钥, 生成 16 轮子密钥 $K_1, K_2 \dots K_{16}$ 。

初始密钥(64bit)经过置换 PC-1, 去掉了 8 个奇偶校验位, 留下 56 bit, 接着分成两个 28 bit 的分组 C_0 与 D_0 , 再分别经过一个循环左移函数 LS_1 , 得到 C_1 与 D_1 , 连成 56 bit 数据, 然后经过置换 PC-2, 输出子密钥 K_1 , 以此类推产生 K_2 至 K_{16} 。

注意: 置换 PC-1、PC-2 会在下文具体给出;

函数 LS_i 为向左循环移动 $\begin{cases} 1 \text{ 个位置}(i=1,2,9,16) \\ 2 \text{ 个位置}(i=3,4,5,6,7,8,10,11,12,13,14,15)。 \end{cases}$

详见下图:



2) 轮变换函数 f :

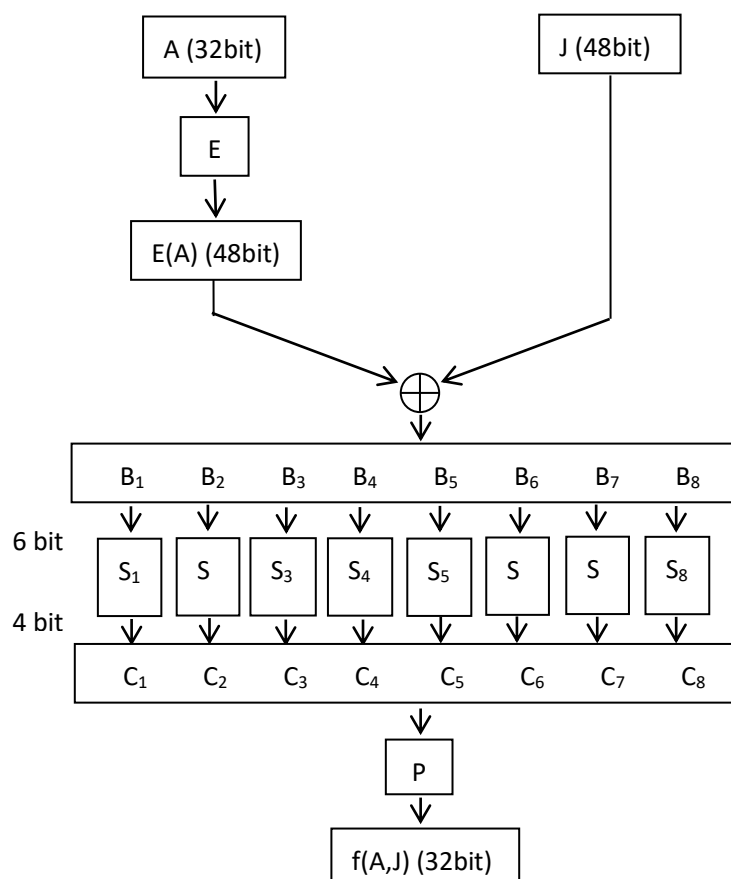
f 是 DES 加解密中每一轮的核心运算, 输入 A (32 bit), J (48 bit), 输出 32 bit。

将 A 做一个扩展运算 E , 变成 48 bit, 记为 $E(A)$ 。计算 $B=E(A) \oplus J$, 将 B 分为 8 组 $B_1 \dots B_8$, 每组 B_i 为 6 bit, 通过相应的 S 盒 S_i , 输出 C_i 为 4 bit, 将所有 C_i

连成 $C(32 \text{ bit})$ ，再通过置换 P ，得到最后的输出 $f(A,J)$ ，为 32 bit 。在加密或解密的第 i 轮， $A = R_{i-1}$ ， $J = K_i$ 。

注意：每个 S 盒 S_i 是 4×16 的矩阵，输入 $b_0b_1b_2b_3b_4b_5$ (6 bit)，令 L 是 b_0b_5 对应的十进制数， n 是 $b_1b_2b_3b_4$ 对应的十进制数，输出矩阵中第 L 行 n 列所对应数的二进制表示。

详见下图：



3) 加/解密：

DES 的加密和解密几乎一样，不同之处在于加密时输入是明文，子密钥使用顺序为 $K_1K_2 \cdots K_{16}$ ；解密时输入是密文，子密钥使用顺序为 $K_{16}K_{15} \cdots K_1$ 。

以加密为例，输入明文分组 64 bit ，先进行一次初始置换 IP ，对置换后的数据 X_0 分成左右两半 L_0 与 R_0 ，根据第一个子密钥 K_1 对 R_0 实行轮变换 $f(R_0, K_1)$ ，将结果与 L_0 作逐位异或运算，得到的结果成为下一轮的 R_1 ， R_0 则成为下一轮的 L_1 。如此循环 16 次，最后得到 L_{16} 与 R_{16} 。可用下列公式简洁地表示：

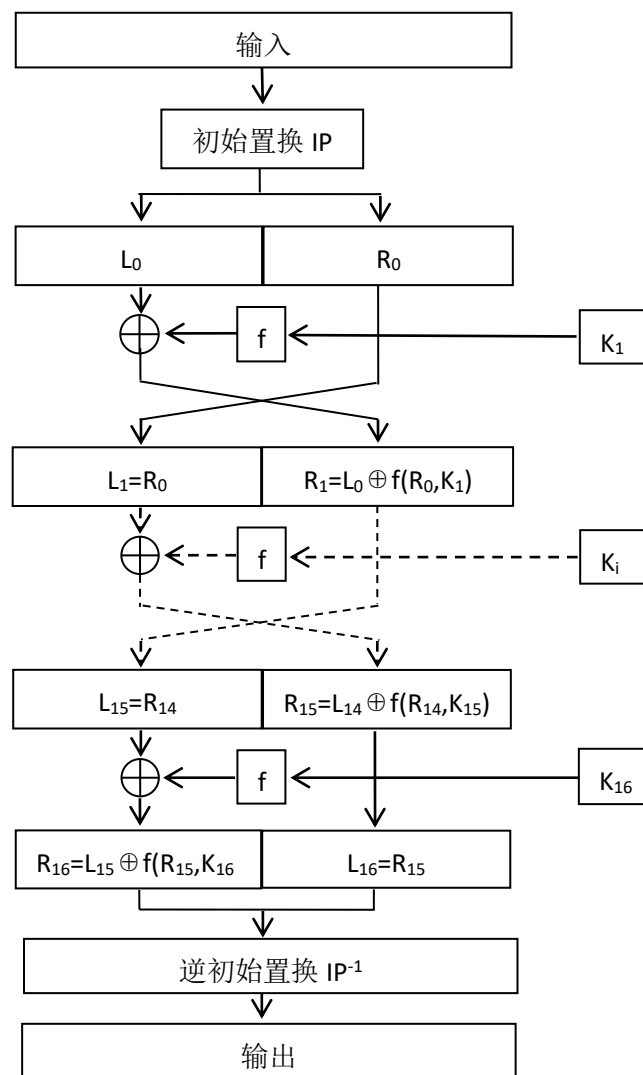
$$R_i = L_{i-1} \oplus f(R_{i-1}, K_i)$$

$$L_i = R_{i-1}, \quad i = 1, 2, \dots, 16$$

最后对 64 bit 数据 R_{16}, L_{16} 实行 IP 的逆置换 IP^{-1} ，即得密文分组。

注意：第 16 轮变换后并未交换 L_{16} 与 R_{16} ，而直接将 R_{16}, L_{16} 作为 IP^{-1} 的输入。

详见下图：



3、使用 Visual C++编写程序，实现 DES 密码及输出界面，
主要步骤：

1) 新建一个空项目，取名 des_test。

2) 在左边的解决方案资源管理器中添加 cpp 文件，取名为 des_test.cpp。

3) 在 des_test.cpp 中先写入

```
#include<iostream>

using namespace std;

static bool SubKey[16][48];    //定义子密钥，使得所有函数都能调用

static const unsigned char IP_Table[64] = {           //定义 IP 置换表
    58,50,42,34,26,18,10,2,60,52,44,36,28,20,12,4,
    62,54,46,38,30,22,14,6,64,56,48,40,32,24,16,8,
    57,49,41,33,25,17,9,1,59,51,43,35,27,19,11,3,
    61,53,45,37,29,21,13,5,63,55,47,39,31,23,15,7};

static const unsigned char IPInv_Table[64] = {        //定义 IP 逆置换表
    40,8,48,16,56,24,64,32,
    39,7,47,15,55,23,63,31,
    38,6,46,14,54,22,62,30,
    37,5,45,13,53,21,61,29,
    36,4,44,12,52,20,60,28,
    35,3,43,11,51,19,59,27,
    34,2,42,10,50,18,58,26,
    33,1,41,9,49,17,57,25};

static const unsigned char E_Table[48] = {           //定义 E 扩展表
    32,1,2,3,4,5,4,5,6,7,8,9,
    8,9,10,11,12,13,12,13,14,15,16,17,
    16,17,18,19,20,21,20,21,22,23,24,25,
    24,25,26,27,28,29,28,29,30,31,32,1};

static const unsigned char P_Table[32] = {           //定义 P 置换表
    16,7,20,21,29,12,28,17,1,15,23,26,5,18,31,10,
    2,8,24,14,32,27,3,9,19,13,30,6,22,11,4,25};

static const unsigned char PC1_Table[56] = {         //定义 PC1 置换表
    57,49,41,33,25,17,9,1,58,50,42,34,26,18,
    10,2,59,51,43,35,27,19,11,3,60,52,44,36,
```

```

63,55,47,39,31,23,15,7,62,54,46,38,30,22,
14,6,61,53,45,37,29,21,13,5,28,20,12,4};

static const unsigned char PC2_Table[48] = {           //定义 PC2 置换表
    14,17,11,24,1,5,3,28,15,6,21,10,
    23,19,12,4,26,8,16,7,27,20,13,2,
    41,52,31,37,47,55,30,40,51,45,33,48,
    44,49,39,56,34,53,46,42,50,36,29,32};

static const unsigned char LS_Table[16] = {           //定义左移位数表
    1,1,2,2,2,2,2,2,1,2,2,2,2,2,1};

static unsigned char S_Box[8][4][16] = {             //S 盒
    //S1
    14,4,13,1,2,15,11,8,3,10,6,12,5,9,0,7,
    0,15,7,4,14,2,13,1,10,6,12,11,9,5,3,8,
    4,1,14,8,13,6,2,11,15,12,9,7,3,10,5,0,
    15,12,8,2,4,9,1,7,5,11,3,14,10,0,6,13,
    //S2
    15,1,8,14,6,11,3,4,9,7,2,13,12,0,5,10,
    3,13,4,7,15,2,8,14,12,0,1,10,6,9,11,5,
    0,14,7,11,10,4,13,1,5,8,12,6,9,3,2,15,
    13,8,10,1,3,15,4,2,11,6,7,12,0,5,14,9,
    //S3
    10,0,9,14,6,3,15,5,1,13,12,7,11,4,2,8,
    13,7,0,9,3,4,6,10,2,8,5,14,12,11,15,1,
    13,6,4,9,8,15,3,0,11,1,2,12,5,10,14,7,
    1,10,13,0,6,9,8,7,4,15,14,3,11,5,2,12,
    //S4
    7,13,14,3,0,6,9,10,1,2,8,5,11,12,4,15,
    13,8,11,5,6,15,0,3,4,7,2,12,1,10,14,9,
    10,6,9,0,12,11,7,13,15,1,3,14,5,2,8,4,
    3,15,0,6,10,1,13,8,9,4,5,11,12,7,2,14,

```

```

//S5
2,12,4,1,7,10,11,6,8,5,3,15,13,0,14,9,
14,11,2,12,4,7,13,1,5,0,15,10,3,9,8,6,
4,2,1,11,10,13,7,8,15,9,12,5,6,3,0,14,
11,8,12,7,1,14,2,13,6,15,0,9,10,4,5,3,
//S6
12,1,10,15,9,2,6,8,0,13,3,4,14,7,5,11,
10,15,4,2,7,12,9,5,6,1,13,14,0,11,3,8,
9,14,15,5,2,8,12,3,7,0,4,10,1,13,11,6,
4,3,2,12,9,5,15,10,11,14,1,7,6,0,8,13,
//S7
4,11,2,14,15,0,8,13,3,12,9,7,5,10,6,1,
13,0,11,7,4,9,1,10,14,3,5,12,2,15,8,6,
1,4,11,13,12,3,7,14,10,15,6,8,0,5,9,2,
6,11,13,8,1,4,10,7,9,5,0,15,14,2,3,12,
//S8
13,2,8,4,6,15,11,1,10,9,3,14,5,0,12,7,
1,15,13,8,10,3,7,4,12,5,6,11,0,14,9,2,
7,11,4,1,9,12,14,2,0,6,10,13,15,3,5,8,
2,1,14,7,4,10,8,13,15,12,9,0,3,5,6,11};

```

4) 编写各个模块函数，例如：

```

void ByteToBit(bool* Out, const unsigned char* In, int bits);
    //将字符数组 In 转化为 bool 数组 Out，bits 控制转换 bit 数
void HalfByteToBit(bool* Out, const unsigned char* In, int bits);
    //将半字符数组 In(4 bit)转化为 bool 数组 Out
void BitToByte(unsigned char* Out, const bool* In, int bits);
    //将 bool 数组 In 转化为字符数组 Out
void Transform(bool* Out, bool* In, const unsigned char* Table, int len);
    //利用置换表 Table 将 bool 数组 In 转换成长度为 len 的 bool 数组 Out
void RotateL(bool* In, int len, int loop);

```

```

//对长度为 len 的 bool 数组 In 循环左移 loop 位
void Des_SetSubKey(unsigned char Key[8]);
//利用初始密钥 Key 生成 16 轮子密钥 SubKey,
//注意前文已定义 static bool SubKey[16][48];
void Xor(bool* InA, bool* InB, int len);
//将 bool 数组 InB 与 InA 逐位做异或, 结果存入 InA
void S_Func(bool Out[32], bool In[48]);
//利用 S 盒 S_Box 将 bool 数组 In 转换为 bool 数组 Out
void F_Func(bool In[32], bool Ki[48]);
//利用子密钥 Ki 对 bool 数组 In 做轮变换函数, 结果仍存入 In
void Des_Run(unsigned char Out[8], unsigned char In[8], bool flag);
//Des 加解密: 输入字符数组 In, 输出字符数组 Out,
//flag=1 时代表加密, flag=0 时代表解密。

```

5) 编写主函数, 调试程序, 目的是测试各个函数编写正确。

6) 所有函数测试都正确后, 新建一个对话框项目 DES, 包含明文框、密钥框、密文框、解密文框等, 以及加密, 解密, 清空按钮, 将以上的置换表、S 盒以及所有子函数声明及实现代码全部复制到 DESDlg.cpp 中的合适位置, 再编写按钮事件, 实现加解密功能。

提示点:

1) 输入输出尽量使用 cin、cout 这两个函数, 编写函数时多输出中间变量查看结果。

2) 函数 memcpy(void *dest, const void *src, size_t n) 用于将源地址 src 处 n 个单位数据拷贝至目的地址 dest 处, 适用于 bool 数组的拷贝。

3) 部分函数参考代码

```

void Transform(bool* Out, bool* In, const unsigned char* Table, int len)
{
    bool Temp[256];           //临时数组长度定义成足够大的即可
    for(int i=0;i<len;i++)
        Temp[i] = In[Table[i]-1];
    memcpy(Out, Temp, len);
}

```



```

}

void ByteToBit(bool* Out, const unsigned char* In, int bits)
{
    for (int i=0;i<bits;i++)
        Out[i] = (In[i/8]>>(7-i%8)) & 1;
}

void HalfByteToBit(bool* Out, const unsigned char* In, int bits)
{
    for (int i=0;i<bits;i++)
        Out[i] = (In[i/4]>>(3-i%4)) & 1;
}

void BitToByte(unsigned char* Out, const bool* In, int bits)
{
    memset(Out, 0, bits/8);
    for(int i=0;i<bits;i++)
        Out[i/8] = (In[i]<<(7-i%8)) + Out[i/8];
}

```

DES 密码程序代码如下：(所有模块函数代码以及按钮事件代码)

//将字符数组 In 转化为 bool 数组 Out，bits 控制转换 bit 数

```

void ByteToBit(bool* Out, const unsigned char* In, int bits)
{
    for (int i = 0; i < bits; i++)
        Out[i] = (In[i / 8] >> (7 - i % 8)) & 1;
}

```

//将半字符数组 In(4 bit)转化为 bool 数组 Out

```

void HalfByteToBit(bool* Out, const unsigned char* In, int bits)
{
    for (int i = 0; i < bits; i++)
        Out[i] = (In[i / 4] >> (3 - i % 4)) & 1;
}

```

//将 bool 数组 In 转化为字符数组 Out

```

void BitToByte(unsigned char* Out, const bool* In, int bits)
{
    memset(Out, 0, bits / 8);
}

```

```

    for (int i = 0; i < bits; i++)
        Out[i / 8] = (In[i] << (7 - i % 8)) + Out[i / 8];
}

```

//利用置换表 Table 将 bool 数组 In 转换成长度为 len 的 bool 数组 Out

```

void Transform(bool* Out, bool* In, const unsigned char* Table, int len)
{
    bool Temp[256] = {0};
    for(int i = 0; i < len; i++)
        Temp[i] = In[Table[i] - 1];
    memcpy(Out, Temp, len);
}

```

//对长度为 len 的 bool 数组 In 循环左移 loop 位

```

void RotateL(bool* In, int len, int loop)
{
    bool Temp[28];
    memcpy(Temp, In, loop);
    for (int i = 0; i < len - loop; i++)
        In[i] = In[i + loop];
    memcpy(In + len - loop, Temp, loop);
}

```

//将 bool 数组 InB 与 InA 逐位做异或，结果存入 InA

```

void Xor(bool* InA, bool* InB, int len)
{
    for (int i = 0; i < len; i++)
        InA[i] ^= InB[i];
}

```

```

//利用 S 盒 S_Box 将 bool 数组 In 转换为 bool 数组 Out
void S_Func(bool Out[32], bool In[48])
{
    for (int i = 0; i < 8; i++)
    {
        int row = (In[i * 6] << 1) + In[i * 6 + 5];
        int col = (In[i * 6 + 1] << 3) + (In[i * 6 + 2] << 2) + (In[i * 6 + 3] << 1)
+ In[i * 6 + 4];
        int val = S_Box[i][row][col];
        for (int j = 0; j < 4; j++)
            Out[i * 4 + j] = (val >> (3 - j)) & 1;
    }
}

```

//利用子密钥 Ki 对 bool 数组 In 做轮变换函数

```

void F_Func(bool In[32], bool Ki[48])
{
    bool Expanded[48];
    Transform(Expanded, In, E_Table, 48);
    Xor(Expanded, Ki, 48);
    S_Func(In, Expanded);
    Transform(In, In, P_Table, 32);
}

```

//生成 16 轮子密钥 SubKey

```

void Des_SetSubKey(unsigned char Key[8])
{
    bool KeyBits[64];
    bool Key56[56];
    bool Ci[28], Di[28];

```

```

    ByteToBit(KeyBits, Key, 64);
    Transform(Key56, KeyBits, PC1_Table, 56);
    memcpy(Ci, Key56, 28);
    memcpy(Di, Key56 + 28, 28);

    for(int i = 0; i < 16; i++)
    {
        RotateL(Ci, 28, LS_Table[i]);
        RotateL(Di, 28, LS_Table[i]);
        bool Combined[56];
        memcpy(Combined, Ci, 28);
        memcpy(Combined + 28, Di, 28);
        Transform(SubKey[i], Combined, PC2_Table, 48);
    }
}

```

//Des 加解密:输入字符数组 In,输出字符数组 Out,flag=1 时代表加密,flag=0 时代表解密

```

void Des_Run(unsigned char Out[8], unsigned char In[8], bool flag)
{
    bool M[64], L[32], R[32], Temp[32];
    ByteToBit(M, In, 64);
    Transform(M, M, IP_Table, 64);
    memcpy(L, M, 32);
    memcpy(R, M + 32, 32);

    for (int i = 0; i < 16; i++) {
        memcpy(Temp, R, 32);
        F_Func(Temp, SubKey[flag ? i : 15 - i]);
        Xor(Temp, L, 32);
    }
}

```

```

        memcpy(L, R, 32);
        memcpy(R, Temp, 32);
    }

    bool Combined[64];
    memcpy(Combined, R, 32);
    memcpy(Combined + 32, L, 32);
    Transform(M, Combined, IPInv_Table, 64);
    BitToByte(Out, M, 64);
}

//打印十六进制
void PrintHex(unsigned char* data, size_t length)
{
    for (size_t i = 0; i < length; i++)
        printf("%02X ", data[i]);
    cout << endl;
}

int main()
{
    string plainTextInput, keyInput;

    //输入明文
    cout << "请输入明文（任意长度字符串）:";
    getline(cin, plainTextInput);

    //将明文分成 8 字节块，并输出分组形式（16 进制）
    size_t blocks = (plainTextInput.size() + 7) / 8;
    unsigned char* plainText = new unsigned char[blocks * 8];

```

```
unsigned char block[8] = {0};
```

```
cout << "明文的分组形式（16 进制）：";
```

```
cout << "\n";
```

```
for (size_t i = 0; i < blocks; i++) {
```

```
    memset(block, 0, 8);
```

```
    for (size_t j = 0; j < 8 && i * 8 + j < plainTextInput.size(); j++)
```

```
        block[j] = plainTextInput[i * 8 + j];
```

```
    PrintHex(block, 8);
```

```
    memcpy(plainText + i * 8, block, 8);
```

```
}
```

```
//输入密钥
```

```
cout << "请输入密钥（任意长度字符串）：";
```

```
getline(cin, keyInput);
```

```
//处理密钥，将其截断或填充为 8 字节并输出分组形式（16 进制）
```

```
unsigned char Key[8] = {0};
```

```
for (size_t i = 0; i < 8; i++)
```

```
    Key[i] = keyInput[i % keyInput.size()];
```

```
cout << "密钥的分组形式（16 进制）：";
```

```
cout << "\n";
```

```
PrintHex(Key, 8);
```

```
//设置子密钥
```

```
Des_SetSubKey(Key);
```

```
//加密
```

```
unsigned char* cipherText = new unsigned char[blocks * 8]();
```

```
for (size_t i = 0; i < blocks; i++) {
```

```

        Des_Run(cipherText + i * 8, plainText + i * 8, true);
    }

    //输出密文
    cout << "密文: ";
    PrintHex(cipherText, blocks * 8);

    //解密
    unsigned char* decryptedText = new unsigned char[blocks * 8]();
    for (size_t i = 0; i < blocks; i++) {
        Des_Run(decryptedText + i * 8, cipherText + i * 8, false);
    }

    //输出解密后的明文
    cout << "解密后的明文: ";
    for (size_t i = 0; i < blocks * 8 && decryptedText[i] != '\0'; i++)
        cout << decryptedText[i];
    cout << endl;

    //清理内存
    delete[] plainText;
    delete[] cipherText;
    delete[] decryptedText;

    return 0;
}

```

DES 密码输出界面截屏如下：

请输入明文（任意长度字符串）： This is DES!

明文的分组形式（16进制）：

54 68 69 73 20 69 73 20

44 45 53 21 00 00 00 00

请输入密钥（任意长度字符串）： dsah

密钥的分组形式（16进制）：

64 73 61 68 64 73 61 68

密文： 1F 0D A9 2D B5 61 22 49 73 1F 95 D7 2C 23 4E A5

解密后的明文： This is DES!